# gomall 后端实战·五讲合订

幂等 / 防超发 / 缓存一致性 / Outbox+Saga / 限流熔断

---

gomall ·后端实战

# 合订本目录

| # | 主题 | 学时 | 合订本起始页 |
|---|---|---|---|
| 01 | 幂等中间件设计 | 60 min | p. 4 |
| 02 | 防超发与抢购库存 | 90 min | p. 33 |
| 03 | 缓存一致性 | 60 min | p. 63 |
| 04 | Outbox 模式与 Saga 补偿 | 90 min | p. 93 |
| 05 | 限流与熔断 | 60 min | p. 123 |

- 全册 5 deck × 29–30 页 = 约 147 页。
- 每份 deck 末 3 页固定为「面试 Q&A（上）/（下）/ 代码位置一览」。
- 推荐顺序：01 → 02 → 03 → 05 → 04（难度递增 + 故事性）。

建议每讲独立放映；连讲请用 `master.pdf` 翻到对应起始页。

# 这套 deck 怎么用

**一节课讲一个主题** 直接放对应 `0?-*.pdf`（29–30 页 / 60–90 min）

**一整门课串讲** 按 01→05 顺序连讲，建议拆 2 天

**单页打印讲义** `pdfjam --nup 2x2 --landscape 0?-*.pdf`

**面试前自查** 打印每份末 3 页（Q&A + 代码位置）当 cheat sheet

编译：`./build.sh`
源码：`docs/slides/`
博客长文：`docs/blog/0?-*.md`
压测数据：`stressTest/REPORT.md`
设计文档：`docs/slides-pipeline/`

# 幂等中间件设计

## 从 Idempotency-Key 到 Redis Lua 状态机

gomall · 后端实战

# 今日大纲

为什么需要幂等

两个常见错法

三态机正解

中间件落地

压测数据

设计取舍

# 为什么需要幂等

# 凌晨两点的电话

- 用户在 App 点了一次” 确认支付”。
- 网络抖动，前端转圈不动，他烦了，连点了三下。
- 第二天客服群炸了：同一笔订单扣了三次款。

> 重复请求只生效一次——这就是幂等。

源自 *docs/blog/01-idempotency.md* § 一个让人崩溃的场景

# 重复请求的四种来源

任何一层

都能制造重复——服务端必须假设” 上游每个请求都可能重发”。

# 两个常见错法

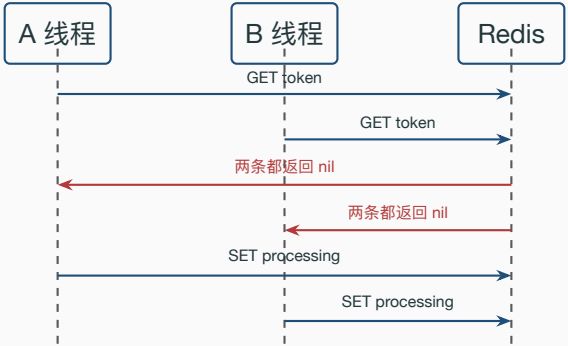
# 错法一：唯一索引硬扛

```sql
CREATE UNIQUE INDEX uk_user_token
  ON orders (user_id, idempotency_token);
```

- 第一次插入成功，第二次冲突 `1062 Duplicate`。
- 但客户端拿不到第一次的响应体——它需要订单号。
- 跨表场景失效：扣库存、写积分、发券都需要独立 token。

对应 *docs/blog/01-idempotency.md § 错误的做法 1*

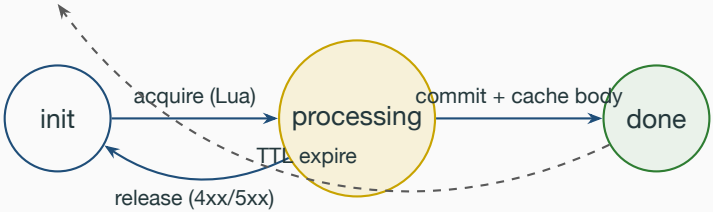
# 错法二：先 GET 再 SET 的竞态




两个线程都判断” 没占用”→ 两次都进入业务 → 还是重复扣款。

# 三态机正解

# 正解：状态机 + Lua + 响应回放




**三种命中**：1 拿锁 / 2 回放

done / 3 处理中拒绝 / 0 token 不存在。

# acquire 四种返回的语义

| 返回码 | 含义 | 服务端响应 |
|---|---|---|
| 1 | init→processing 成功 | 进入业务逻辑 |
| 2 | 命中 done，附 result | 回放缓存的响应体 |
| 3 | 命中 processing | 返回” 处理中”，让客户端等待 |
| 0 | token 不存在 | 返回”token 无效或过期” |

四种返回必须在一段 Lua 里判定，否则就是” 先 GET 再 SET” 的换皮。

*repository/cache/idempotency.go:32–50*

# Lua 脚本完整实现

Listing 1: *repository/cache/idempotency.go:32–50*

```lua
local v = redis.call('HGET', KEYS[1], 'state')
if v == false or v == nil then
   return {0, ''}
end
if v == 'done' then
   local r = redis.call('HGET', KEYS[1], 'result')
   return {2, r or ''}
end
if v == 'processing' then
   return {3, ''}
end
if v == 'init' then
   redis.call('HSET', KEYS[1], 'state', 'processing')
   redis.call('EXPIRE', KEYS[1], tonumber(ARGV[1]))
   return {1, ''}
end
return {0, ''}
```

# 为什么必须 Lua

- Redis 单线程串行执行命令，但 GET + SET 是**两条独立命令**，中间会被其他客户端插入。
- Lua 脚本对外是一条” 原子命令”：所有 KEYS 操作在脚本运行期间不会被打断。
- 写 Lua 还有一个好处：把” 返回码语义” 封进脚本，调用方只关心 1/2/3/0。

Lua $\neq$ 分布式锁，它是把”check-then-act” 装进一条原子命令的工具。

# 中间件落地

# 用户隔离的 key 命名

Listing 2: *repository/cache/key.go:15–21*

```go
const IdempotencyKey = "idemp:%d:%s"

func IdempotencyTokenKey(userId uint, token string) string {
    return fmt.Sprintf(IdempotencyKey, userId, token)
}
```

不用 `idemp:{token}`：避免恶意用户预测 token 抢占别人的请求，也方便按用户清理。

# responseRecorder：拿到 gin 的响应体

Listing 3: *middleware/idempotency.go:18–31*

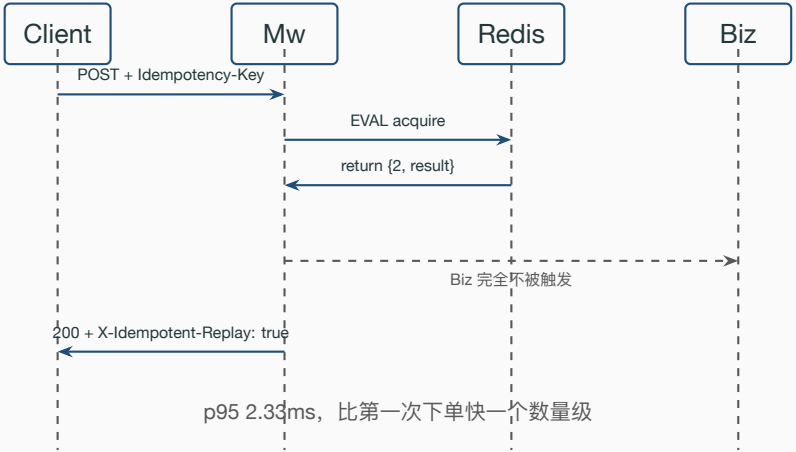
```go
18 type responseRecorder struct {
19    gin.ResponseWriter
20    body *bytes.Buffer
21 }
22
23 func (r *responseRecorder) Write(b []byte) (int, error) {
24    r.body.Write(b)
25    return r.ResponseWriter.Write(b)
26 }
27
28 func (r *responseRecorder) WriteString(s string) (int, error) {
29    r.body.WriteString(s)
30    return r.ResponseWriter.WriteString(s)
31 }
```

# 中间件主干

Listing 4: *middleware/idempotency.go:60–111*

```go
key := cache.IdempotencyTokenKey(user.Id, token)
state, cached, err := cache.AcquireIdempotencyLock(c.Request.Context(), key)
switch state {
case 2: // done -> 回放
   c.Header("X-Idempotent-Replay", "true")
   c.Data(http.StatusOK, "application/json", []byte(cached))
   c.Abort(); return
case 3: c.JSON(200, gin.H{"msg":"处理中"}); c.Abort(); return
case 0: c.JSON(200, gin.H{"msg":"token 无效"}); c.Abort(); return
}
recorder := &responseRecorder{ResponseWriter: c.Writer, body: bytes.NewBuffer(nil)}
c.Writer = recorder
c.Next()
if len(c.Errors) > 0 || recorder.Status() >= 400 {
   cache.ReleaseIdempotencyLock(c.Request.Context(), key); return
}
cache.CommitIdempotencyResult(c.Request.Context(), key, recorder.body.String())
```

# 第二次请求的完整时序


Client
Mw
Redis
Biz
POST + Idempotency-Key
EVAL acquire
return {2, result}
Biz 完全不被触发
200 + X-Idempotent-Replay: true
p95 2.33ms，比第一次下单快一个数量级

# 失败回滚 vs 永久死锁

- 业务返回 4xx/5xx 时 `ReleaseIdempotencyLock` 将状态打回 `init`。
- 客户端拿同一 token 重试时，可以重新进入 `processing`。
- 若失败也写 `done`：客户端重试只能拿到上次的错误响应，永远卡死。

processing → init 是状态机里最容易被忽视的一条边。

*middleware/idempotency.go:101–105 + repository/cache/idempotency.go:87–90*

# 压测数据

# 50 VU × 15s = 755,033 请求

| 指标 | 数值 |
|---|---|
| 压测脚本 | `idempotency_replay.js` |
| VU 数 | 50 |
| 时长 | 15s |
| 总请求 | **755,033** |
| DB 实际订单 | **1** |

750K 个 POST，DB 落 1 笔——这是中间件应有的”过滤强度”。

*stressTest/REPORT.md §4.* 幂等中间件

# 延迟分布：回放比真实下单快 5 倍

| 链路 | RPS | p50 | p95 | max |
|---|---|---|---|---|
| Ping 基线 | 64,254 | 0.91ms | 3.51ms | 109ms |
| 幂等下单 (回放) | **50,319** | 0.50ms | **2.33ms** | 73ms |

回放路径只走 Redis Hash + Lua，避免了下单完整链路里的事务、库存预扣、Outbox 写入。*stressTest/REPORT.md § 链路压测表第 1 行 + 第 4 行*

# 设计取舍

# TTL 选 5 分钟，怎么定?

- 太短：客户端重试网络往返 1–2s，token 提前过期会让重试变成新请求。
- 太长：占用 Redis 内存；如果 token 泄漏，攻击者能长期复用。
- 5 分钟：覆盖单次业务的所有重试窗口，过期后客户端必须重新申请。

*repository/cache/idempotency.go:16–17* `IdempotencyTokenTTL = 5 * time.Minute`

# Redis 挂了，怎么 fail?

- 选 fail-close：Lua 报错时返回 5xx，**不放行**业务。
- 选 fail-open：Lua 报错继续走业务——等于关闭幂等保护。
- 钱相关接口必须 fail-close，宁可短时全断也不能漏一笔重复扣款。

> 安全默认 (safe default) = 失败方向选” 对用户负责” 的那一边。

*middleware/idempotency.go:61–69*

# 幂等 vs 分布式锁 vs 去重表

| 机制 | 解决的问题 | 典型用法 |
| --- | --- | --- |
| 幂等中间件 | 重复**请求**只生效一次 | 下单/支付/退款 |
| 分布式锁 | 同一资源同时**只有一个执行者** | 定时任务、库存合并扣减 |
| 去重表 | 已处理的**业务 id** 不再处理 | MQ 消费端、回调接收 |

三者都关心”避免重复”，但作用维度不同——面试时一定要区分清楚。

# 面试 Q&A（上）

**Q1**：为什么” 先 GET 再 SET” 不是幂等?

A：两条命令之间存在竞态，必须用 Lua 原子化。

**Q2**：客户端没拿到响应就重试，怎么避免重复扣款?

A：第二次进入命中 done，回放上一次缓存的响应体，业务不再执行。

**Q3**：幂等 token 的生命周期谁来负责？TTL 设多长?

A：服务端发 token + 设 TTL；5 分钟覆盖单次业务的重试窗口。

# 面试 Q&A（下）

**Q4**：业务处理失败，token 状态怎么回滚？

A：processing → init，让同 token 还能重试；不能写 done 否则永久卡死。

**Q5**：幂等 vs 分布式锁的边界？

A：幂等让同一请求只生效一次；锁让同一资源同时只有一个执行者。

**反追问**：跨方法能共用 token 吗？`Idempotency-Key` 是不是规范？

A：不建议跨方法共用；这个 Header 名是 IETF 草案 `draft-ietf-httpapi-idempotency-key-header`。

# 代码位置一览

**中间件主干** *middleware/idempotency.go:35–112*

**Lua 脚本** *repository/cache/idempotency.go:32–50*

**acquire 包装** *repository/cache/idempotency.go:64–76*

**commit / release** *repository/cache/idempotency.go:79–90*

**key 命名** *repository/cache/key.go:19–21*

**压测脚本** *stressTest/idempotency_replay.js*

```
复现：k6 run stressTest/idempotency_replay.js
```

# 防超发与抢购库存

## Redis 原子扣减 vs `SELECT FOR UPDATE` 的实战对比

gomall · 后端实战

# 今日大纲

超发的代价

方案矩阵

DB 悲观锁

Redis Lua 双 key

压测对比

进阶：双桶库存

# 超发的代价

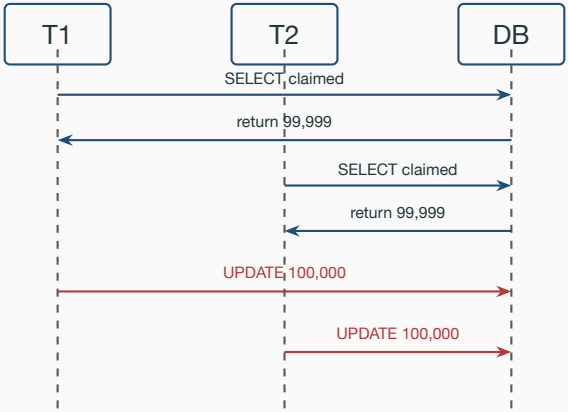
# 618 那一夜

- 配额：10 万张满 200 减 50 优惠券。
- 凌晨开抢，3 分钟全光，运营群一片欢呼。
- 财务次日对账发现：实际发出 23 万张。

> 超发 13 万 × 50 元 = 650 万。一行代码值半个 P0。

源自 *docs/blog/02-anti-oversell.md* § 优惠券超发能赔多少钱

# check-then-act 的经典时序




两个事务都看到 99,999，都觉得” 还能发一张” → 超发。

# 错法：单条 SQL 看似”原子”

```sql
UPDATE coupon_batch
  SET claimed = claimed + 1
 WHERE id = ? AND claimed < total;
```

- 看起来一条原子 UPDATE，”行锁会兜底”。
- 但还要写 `user_coupon` 记录、校验 per-user 配额——多条 SQL 的原子性回到了零。
- 一旦套上事务，并发下还是要靠 row lock 串行化，吞吐塌方。

对应 *docs/blog/02-anti-oversell.md § 看起来天衣无缝*

# 方案矩阵

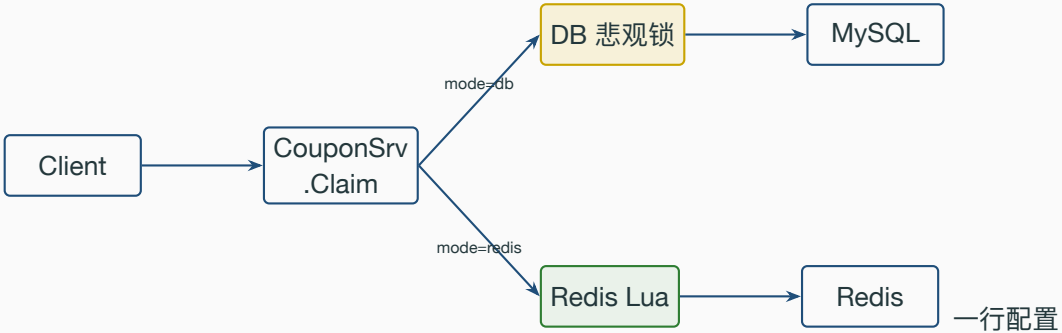
# 五种防超发方案

| 方案 | 一致性 | 吞吐 | 复杂度 | 适用 |
|---|---|---|---|---|
| 悲观锁 `FOR UPDATE` | 强 | 低 | 低 | 低频/账务 |
| 乐观锁 `version` | 强 | 中 | 中 | 中频/CAS 友好 |
| Redis `DECR` 单命令 | 强 | 高 | 低 | 单一计数 |
| Redis Lua 多 key | 强 | 高 | 中 | **多条件抢资源** |
| Redlock 分布式锁 | 强 | 中 | 高 | 跨资源/慎用 |

本项目实现了两种：

**Redis Lua** (常用) 和 `FOR UPDATE` (对比基线)。*service/coupon.go:65–99 mode 切换*

# 两种模式可热切换




`mode=db` 或 `mode=redis` 切换两套实现，便于压测对比。*service/coupon.go:65–99*

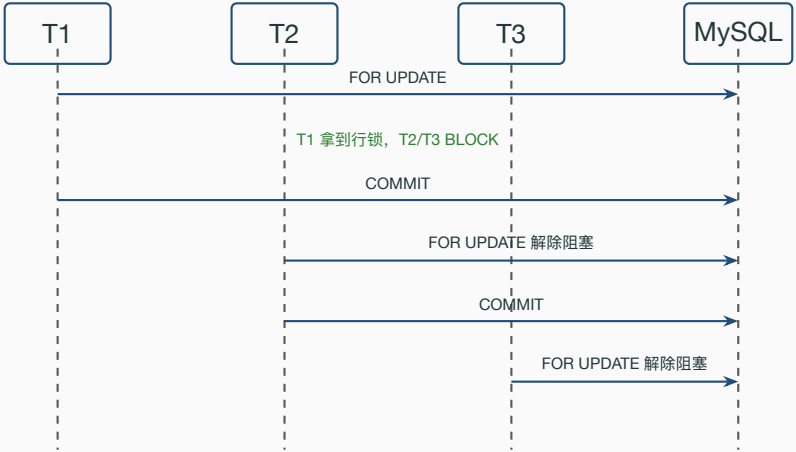
# DB 悲观锁

# SELECT FOR UPDATE 加 count 校验

Listing 1: *repository/db/dao/coupon.go:41–85*

```go
err := d.Transaction(func(tx *gorm.DB) error {
    var batch model.CouponBatch
    if err := tx.Set("gorm:query_option", "FOR UPDATE").
        Where("id = ?", batchId).First(&batch).Error; err != nil { return err }
    if batch.Claimed >= batch.Total { return errors.New("已抢光") }
    var owned int64
    tx.Model(&model.UserCoupon{}).Where("user_id = ? AND batch_id = ?",
        userId, batchId).Count(&owned)
    if owned >= batch.PerUser { return errors.New("超出单人上限") }
    tx.Model(&model.CouponBatch{}).Where("id = ?", batch.ID).
        Update("claimed", gorm.Expr("claimed + 1"))
    return tx.Create(uc).Error
})
```

# DB 锁的原理：行锁串行化




正确性满分——但 N 个并发就是 N 倍延迟，DB 单点就是上限。

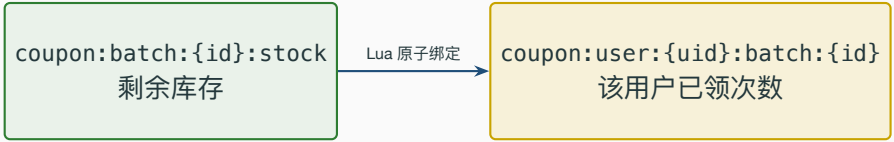
# DB 锁的代价

- 正确性强：行锁让整个 batch 真正串行。
- 吞吐上限：实测 RPS 50,142，**max 延迟 453ms**（尾延迟暴雷）。
- 索引选错时会从行锁升级为间隙锁甚至表锁。
- DB 是单点，扩容代价高于 Redis。

*stressTest/REPORT.md §6. 优惠券 Redis vs DB 锁*

# Redis Lua 双 key

# 双 key 设计




- 一段 Lua 里同时检查” 还有库存”+” 用户没超额”。
- 任何一步失败都不会改另一边——这是双 key 必须 Lua 的本质。

*repository/cache/coupon.go:12–44*

# 抢券 Lua 完整脚本

Listing 2: *repository/cache/coupon.go:30–44*

```lua
local stock = tonumber(redis.call('GET', KEYS[1]))
if stock == nil or stock <= 0 then
   return -1
end
local owned = tonumber(redis.call('GET', KEYS[2]))
if owned == nil then owned = 0 end
if owned >= tonumber(ARGV[1]) then
   return -2
end
redis.call('DECR', KEYS[1])
redis.call('INCR', KEYS[2])
redis.call('EXPIRE', KEYS[2], tonumber(ARGV[2]))
return 1
```

返回 1 = 成功；$-1$ = 已抢光；$-2$ = 超出单人上限。

# 为什么这能保证不超发

- Redis 是单线程：Lua 脚本被当作一条命令，期间不接受其他客户端的命令。
- `DECR` 是真正的原子计数器；不可能” 两个客户端同时把库存从 1 减到 -1”。
- 多条 `redis.call` 一起原子化——不需要 WATCH/MULTI。

并发性来自客户端并行；防超发的根基是 Redis 服务端串行。

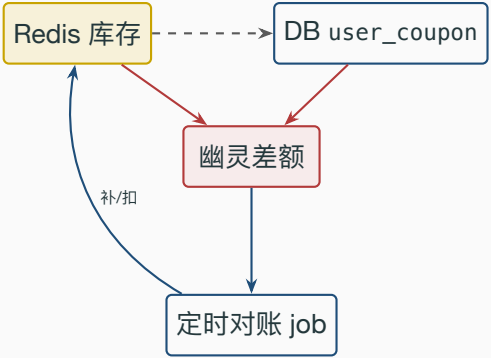
# 落库失败要回滚 Redis

Listing 3: *service/coupon.go:83–98*

```go
ok, err := cache.ClaimCouponAtomic(ctx, u.Id, batchId, batch.PerUser)
if err != nil { return nil, err }
if !ok { return nil, errors.New("领取失败") }

uc, err := dao.NewCouponDao(ctx).PersistClaim(u.Id, batchId, batch.ValidDays)
if err != nil {
    // 落库失败回滚 redis，避免库存幽灵消耗
    cache.RollbackCouponStock(ctx, u.Id, batchId)
    return nil, err
}
return uc, nil
```

# 回滚也可能失败：定时对账兜底




”Redis 扣成功 + DB 写失败 + 回滚也失败” 是真实存在的低概率事件；定时对账是终极兜底。

# 单测：500 抢 100 零超发

Listing 4: *repository/cache/coupon_test.go:11–46*

```go
const total = 100
const concurrency = 500
InitCouponStock(ctx, batchId, total, time.Minute)
var success int64; var wg sync.WaitGroup
for i := 1; i <= concurrency; i++ {
   wg.Add(1); uid := uint(i)
   go func() {
      defer wg.Done()
      ok, _ := ClaimCouponAtomic(ctx, uid, batchId, 1)
      if ok { atomic.AddInt64(&success, 1) }
   }()
}
wg.Wait()
if got := atomic.LoadInt64(&success); got != total {
   t.Fatalf("oversell or undersell: want %d, got %d", total, got)
}
```

# 压测对比

# Redis Lua vs DB 锁：尾延迟差 3 倍

| 方案 | RPS | avg | p95 | **max** |
|---|---|---|---|---|
| Redis Lua | 51,362 | 1.24ms | 3.52ms | **136ms** |
| DB 悲观锁 | 50,142 | 1.29ms | 3.65ms | **453ms** |

*stressTest/REPORT.md § 链路压测表第 5/6 行 + §6*

RPS 接近：因为 PerUser=1 让单用户两边都被卡在 1 张。
关键差异在尾延迟：DB 的 **max 453ms** 暴露了行锁排队的真实成本。

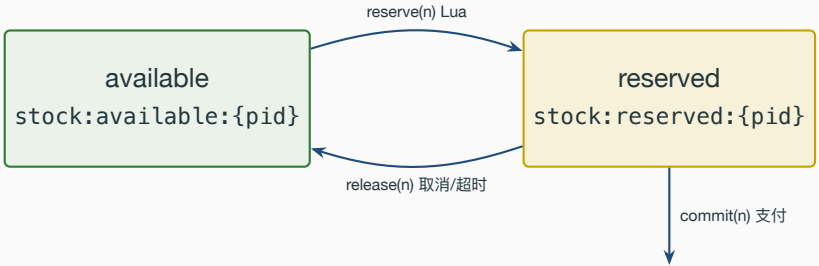
# 选型直觉

抢资源类必须 Redis Lua；DB 锁只配低频高一致场景。

- 秒杀、抢券、限购：Redis Lua（毫秒级，可水平扩容）。
- 账务转账、对账冲销：DB 事务 + 行锁（最终落 DB，逻辑统一）。
- 跨多个资源：考虑 Saga（见 Deck 4），别堆 Redlock。

# 进阶：双桶库存

# 下单场景的 available / reserved




真正消耗 (支付成功)

*repository/cache/inventory.go:10–75*

# reserve / commit / release 三脚本

Listing 5: *repository/cache/inventory.go:78–125*

```go
// 预扣: available -= n, reserved += n
func ReserveStock(ctx context.Context, productID uint, n int64) error

// 支付成功: reserved -= n
func CommitReservation(ctx context.Context, productID uint, n int64) error

// 取消/超时: reserved -= n, available += n
func ReleaseReservation(ctx context.Context, productID uint, n int64) error
```

三段 Lua 守护一个不变量：`available + reserved + sold == 初始库存`。

# 设计决策清单

- 库存源：DB 是真相之源，Redis 是高并发缓冲。
- PerUser 限制：用独立的 user 计数器，不要塞进库存 key。
- Redis 重启：用 RDB/AOF + 启动时从 DB 重灌。
- TTL 设到活动结束 + 24h，留出对账窗口。
- Lua 用 `NewScript` 缓存 SHA，避免每次都传完整脚本。

# 面试 Q&A（上）

**Q1**：100 张券，500 并发，怎么保证不超发？

A：Lua 脚本一次性原子执行”check + DECR”，Redis 单线程做兜底。

**Q2**：Redis 扣成功但 DB 落库失败，怎么办？

A：立即 `RollbackCouponStock` 回滚；万一回滚也失败靠定时对账兜底。

**Q3**：`SELECT FOR UPDATE` 用错索引会怎么样？

A：走非唯一索引或全表扫，行锁可能升级成 GAP 锁甚至表锁，吞吐塌方。

# 面试 Q&A（下）

**Q4**：为什么不能 `if stock>0 then DECR` 两条命令？
A：客户端两次 RTT 之间会被插队，跟” 先 GET 再 SET” 一样竞态。

**Q5**：热点商品怎么分片库存？
A：按 hash 分 N 个子桶，每个桶独立 DECR；空桶时漂到其他桶 (有序兜底)。

**反追问**：能用 Redlock 替代 Lua 吗？
A：不该。Redlock 用来跨资源协调；单 key 抢资源用 Lua 又快又稳。

# 代码位置一览

| | |
|---|---|
| **抢券 Lua** | *repository/cache/coupon.go:30–44* |
| **DB 悲观锁** | *repository/db/dao/coupon.go:41–85* |
| **mode 切换** | *service/coupon.go:65–99* |
| **双桶库存** | *repository/cache/inventory.go:30–75* |
| **500 并发单测** | *repository/cache/coupon_test.go:11–46* |
| **压测脚本** | *stressTest/coupon_claim_redis.js + coupon_claim_db.js* |

复现：`k6 run stressTest/coupon_claim_redis.js`

# 缓存一致性

## Cache Aside、延迟双删与 SETNX 回源锁

gomall · 后端实战

# 今日大纲

问题本质

候选方案对比

单删为什么不够

延迟双删

读路径的击穿保护

binlog 订阅

压测数据

# 问题本质

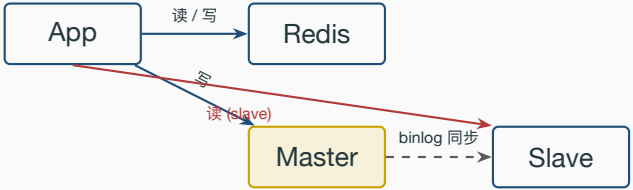
# 你以为很简单的题

”先更新 DB 再删缓存”对吗？
——这句话能挂掉半数中级候选人。

- 表面看：读时回填，写时删除，逻辑闭环。
- 实际：主从延迟 + 网络抖动让”删除”和”回填”的相对顺序无法约束。
- 缓存一致性的本质是：**两份数据，不可能在所有时刻都相等。**

*docs/blog/03-cache-consistency.md § 你以为很简单的题*

# 问题的本质：三个组件 + 主从延迟




写主、读从、删缓存——三条路径在并发下的相对顺序是不可控的。

# 候选方案对比

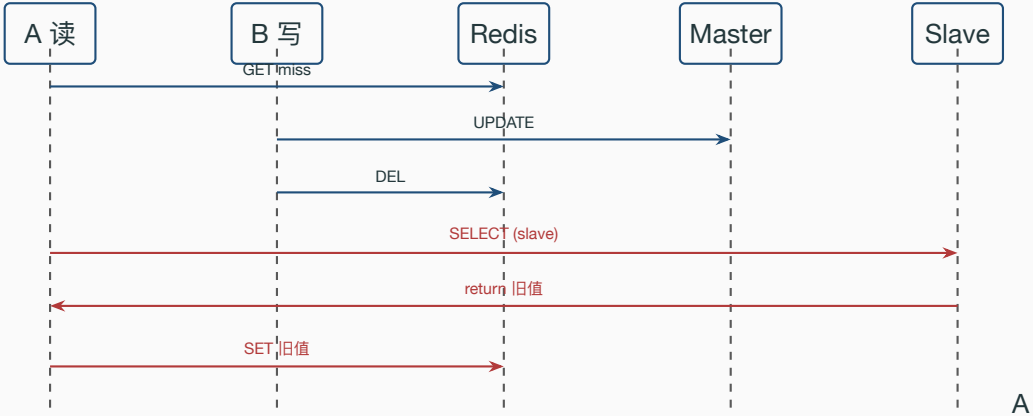
# 四种策略五分钟扫盲

| 策略 | 一致性 | 实现复杂度 | 主要风险 |
|---|---|---|---|
| 先写 DB 再删缓存 | 弱 | 低 | 并发读回填脏值 |
| 先删缓存再写 DB | 弱 | 低 | 并发读回填脏值 |
| 延迟双删 | 中 (窗口可控) | 中 | 窗口取舍 |
| 双写 DB+Cache | 中 | 中 | 部分失败要补偿 |
| binlog → MQ → 刷缓存 | 强 (最终) | 高 | 运维成本 |

*docs/blog/03-cache-consistency.md § 候选方案大评比*

# 单删为什么不够

# 单删时序：四步制造一次脏缓存



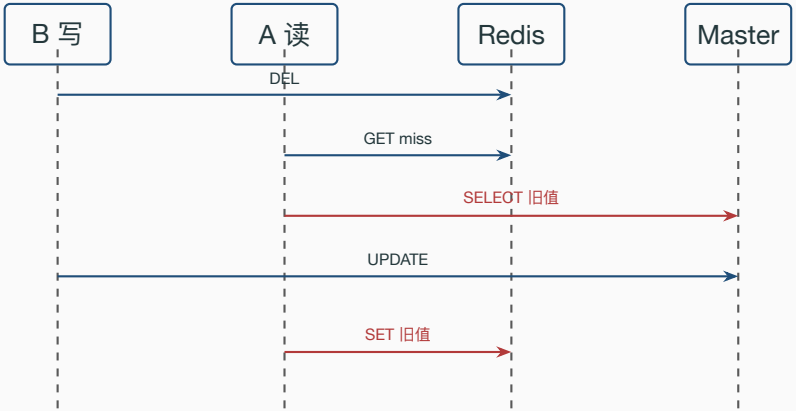

的读发生在 B 删除之前，但 SET 回填发生在 DEL 之后 → 脏值永驻。

# race 的精妙之处

- A 读：从 **slave** 拿到旧值。
- B 写：写 **master** 后 DEL Redis。
- A 把”slave 旧值” 回填进 Redis ——写顺序看似没问题，但读源不同。
- 真正的杀手是主从延迟，而不是” 先写再删” 四个字。

并发读写 + 主从延迟 = 缓存脏数据的根因。

# 反过来” 先删再写” 也不行




窗口换位置但没消失

——” 先删再写” 只是错觉。

# 延迟双删

# 四步延迟双删

- 第一次 DEL 把所有进行中的读”清零”。
- UPDATE 写 master。
- sleep 覆盖主从延迟 + 并发读”回填脏值”的窗口。
- 第二次 DEL 清除可能被脏值回填的缓存。

# 500ms 怎么来的?

- 主从延迟 P99 一般 50–200ms（机房内）。
- 业务读耗时 + 网络 RTT 一般 10–100ms。
- 500ms 给两者留足冗余——不是公式，是经验值。
- 真实系统建议监控主从延迟，动态调整。

> 延迟双删**不是**强一致，是”窗口期可控的最终一致”。

*repository/cache/product.go:16* `ProductDelayInterval = 500 * time.Millisecond`

# ProductUpdate 的写路径

Listing 1: *service/product.go:271–292*

```go
271 func (s *ProductSrv) ProductUpdate(ctx context.Context, req *types.ProductUpdateReq) (...) {
272     product := &model.Product{...}
273     _ = cache.DelProductDetail(ctx, req.ID) // 1. 第一次删
274     err = dao.NewProductDao(ctx).UpdateProduct(req.ID, product) // 2. 写 DB
275     if err != nil {
276         log.LogrusObj.Error(err); return
277     }
278     cache.DoubleDeleteAsync(req.ID, 0) // 3+4. 异步等 500ms 再删
279     emitProductChanged(ctx, req.ID, "update")
280     return
281 }
```

# DoubleDeleteAsync 实现

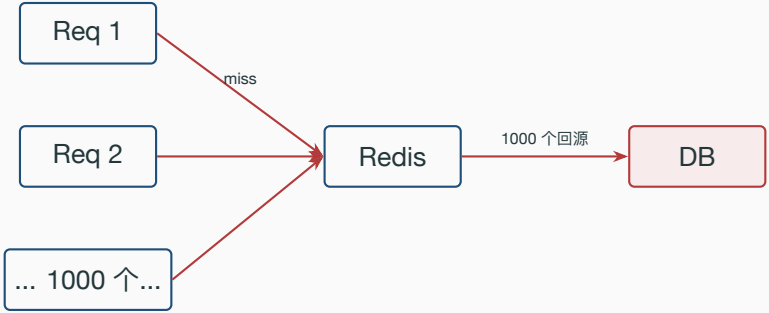
Listing 2: *repository/cache/product.go:64–74*

```go
// DoubleDeleteAsync: 写库后已经做了第一次删除，这里在 interval 后再删一次，
// 用于覆盖"读旧值的并发请求把旧值塞回缓存"的窗口。
func DoubleDeleteAsync(id uint, interval time.Duration) {
    if interval <= 0 {
        interval = ProductDelayInterval
    }
    go func() {
        time.Sleep(interval)
        _ = RedisClient.Del(context.Background(), ProductDetailKey(id)).Err()
    }()
}
```

用独立 goroutine 异步处理，不阻塞业务返回；第二次 DEL 失败也不影响主路径。

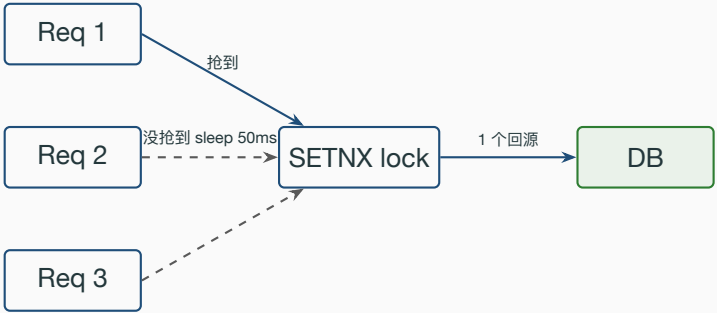
# 读路径的击穿保护

# 读路径的击穿场景




热 key 过期瞬间，所有读全部 miss → 同时打 DB → DB 被打挂。

# SETNX 回源锁：单飞回源




全局只有一个回源；其他请求 sleep 后重读缓存（多半已经命中）。*repository/cache/product.go:55–62 + service/product.go:42–67*

# ProductShow Cache Aside + SETNX

Listing 3: *service/product.go:42–68*

```go
cached := &types.ProductResp{}
if cacheErr := cache.GetProductDetail(ctx, req.ID, cached); cacheErr == nil {
    return cached, nil
}
locked, _ := cache.TryProductLock(ctx, req.ID)
if !locked {
    time.Sleep(50 * time.Millisecond)
    if cacheErr := cache.GetProductDetail(ctx, req.ID, cached); cacheErr == nil {
        return cached, nil
    }
} else {
    defer cache.UnlockProduct(ctx, req.ID)
}
pResp, err := s.loadProductFromDB(ctx, req.ID)
if err != nil { return nil, err }
cache.SetProductDetail(ctx, req.ID, pResp)
return pResp, nil
```

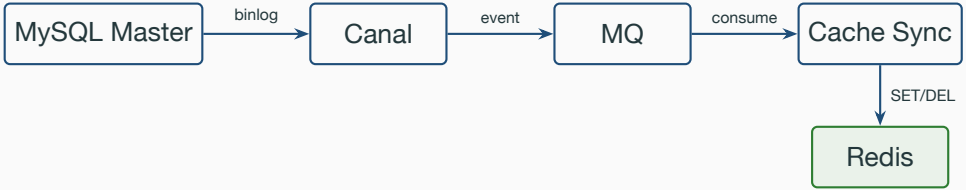
# 为什么不用 Go 的 singleflight

- Go 标准库 `singleflight` 只在**单进程内**合并请求。
- 服务多实例部署后，每个实例自己 singleflight → N 个回源同时打 DB。
- Redis SETNX 是**跨进程**的全局锁，才是真正的” 全局单飞”。
- 短 TTL (3s) 防止锁持有者崩溃导致死锁。

*repository/cache/product.go:14–17* `ProductLockTTL = 3 * time.Second`

# binlog 订阅

# 终极方案：Canal → MQ → 异步刷




- 应用层不再负责缓存一致性，binlog 是事实之源。
- 收益：业务代码简洁；代价：多一套运维 (Canal/Debezium + MQ)。

# 还有哪些坑

- 删失败要重试——引入” 删失败补偿队列”。
- 多层缓存 (本地 + Redis) ——本地缓存的失效要广播。
- TTL 雪崩——给 TTL 加随机抖动 (`ttl + rand[0, 60s]`)。
- 缓存穿透——Bloom Filter / 空值缓存。

*docs/blog/03-cache-consistency.md § 还有哪些坑*

# 压测数据

# 大表场景下的真实收益

| 链路 | RPS | p95 |
|---|---|---|
| Ping 基线 | 64,254 | 3.51ms |
| 商品详情 (DB + PK，未接缓存) | 62,226 | 3.01ms |
| 商品列表 (DB count 全表，956K 行) | **24.5** | **2.50s** |

`product/show` 走主键查询，单点延迟和 ping 几乎重合：硬件 cap 在这里。
`product/list` 跌到 **24.5 RPS / p95 2.5s** ——瓶颈是 956K 行 `COUNT(*)` 每次全表扫，正是缓存能挡的工作量。*stressTest/REPORT.md § 链路压测第 1/2/8 行（基线 64,254 / DB+PK 62,226 / count 全表 24.5 RPS）*

# 游标分页 + 首页缓存：实测 7000× 吞吐

| 订单列表链路 | RPS | p95 |
|---|---|---|
| 旧版深分页 (`OFFSET 1999999 LIMIT 20`，6M 行) | **8.3** | **15.95s** |
| 游标分页 + 5min 首页缓存 (PR #38) | **58,406** | **5.0ms** |
| 吞吐比 | | **≈7,000×** |
| p95 延迟比 | | **≈3,200×** |

优化点有两个：**游标分页**（`WHERE id < last_id LIMIT 20`，PK 倒序 O(1)）+ **首页 5min Redis 缓存**（Cache Aside）。
缓存挡住的不是延迟，是 **DB 全表扫的负载**。这是 Cache Aside 在大表上的真实代差。

*stressTest/REPORT.md §2. 订单列表 + PR #38*

# 面试 Q&A（上）

**Q1**：双写一致性的几种方案，各自取舍？

A：先写后删 / 先删后写都有竞态；延迟双删窗口可控；binlog 异步刷强一致但运维重。

**Q2**：延迟双删的” 延迟” 该设多少？

A：覆盖主从延迟 + 业务读耗时；本项目 500ms 是经验值，建议监控驱动。

**Q3**：缓存击穿和缓存穿透的区别？

A：击穿是热 key 失效瞬间全打 DB；穿透是查不存在的 key 永远不命中缓存。

# 面试 Q&A（下）

**Q4**：为什么删不更新缓存？

A：写双份会引入新的竞态（写顺序），删除是单边操作更鲁棒；下次读会回填。

**Q5**：TTL 雪崩怎么避免？

A：TTL 加随机抖动；热 key 永不过期 + 后台 refresh；多级缓存分散过期时间。

**反追问**：本地缓存 (caffeine) 失效广播怎么实现？

A：用 Redis pub/sub 或 binlog 通道把失效事件广播到所有实例。

# 代码位置一览

**读路径 + SETNX** *service/product.go:42–68*

**写路径 + 双删** *service/product.go:271–292*

**DoubleDeleteAsync** *repository/cache/product.go:64–74*

**TryProductLock / SETNX** *repository/cache/product.go:55–62*

**TTL 常量** *repository/cache/product.go:14–17*

**压测** *stressTest/product_show.js*

```
复现：k6 run stressTest/product_show.js
```

# Outbox 模式与 Saga 补偿

从 DB 事务能不能套住 MQ，到一次真实线上事故

gomall · 后端实战

# 今日大纲

从一个库存虚高 bug 说起

双桶状态机

然而还有双写问题

Outbox 模式

Saga 与故障演练

真实事故复盘

# 从一个库存虚高 bug 说起

# 盘代码时盯到一对孪生函数

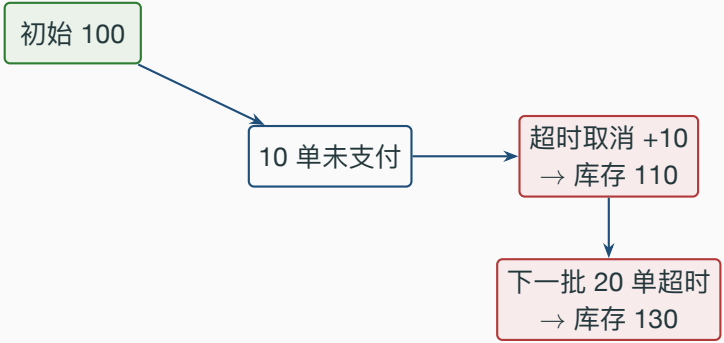
```
// OrderCreate：扣库存 → 写订单
// 旧实现：直接 UPDATE product SET num = num - n

// CancelUnpaidOrder：未付款超时
// 旧实现：UPDATE product SET num = num + n（加回来）
```

- 取消方做了 +n，但下单方早期版本根本没 -n。
- 一对不对称的兄弟函数——库存只增不减。

*service/order_cancel.go:14–63 + service/order.go:40–102*

# 推演结果：库存只增不减




这不是单一 bug，是**库存状态机没设计**：缺少 reserved 中间态。

# 修这个 bug 的两条路

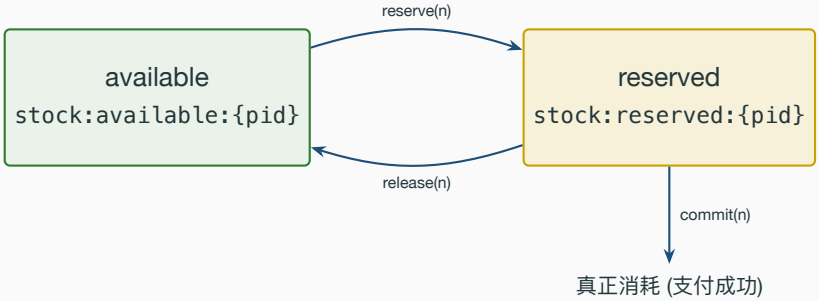
| 方案 | 改动量 | 评价 |
| --- | --- | --- |
| A. 加 `reserved_num` 字段补丁 | 小 | 治标不治本，状态机还是隐式 |
| B. 重设双桶库存状态机 | 中 | **推荐**：显式区分 available / reserved |

选 B：

把”占用”和”消耗”分开，回退路径变得自然。*docs/blog/04-outbox-saga.md § 修这个 bug 的两条路*

# 双桶状态机

# available / reserved / 真正消耗




不变量：`available + reserved + sold = 初始库存`。

*repository/cache/inventory.go:10–75*

# Reserve Lua: available → reserved

Listing 1: *repository/cache/inventory.go:32–44*

```lua
local avail = redis.call('GET', KEYS[1])
if avail == false then
   return -2
end
local need = tonumber(ARGV[1])
if tonumber(avail) < need then
   return -1
end
redis.call('DECRBY', KEYS[1], need)
redis.call('INCRBY', KEYS[2], need)
return 1
```

# Commit & Release Lua

Listing 2: *repository/cache/inventory.go:48–66*

```lua
-- commit: 支付成功, reserved -= n
local r = redis.call('GET', KEYS[1])
if r == false or tonumber(r) < tonumber(ARGV[1]) then
   return -1
end
redis.call('DECRBY', KEYS[1], ARGV[1])
return 1

-- release: 取消/超时, reserved -= n, available += n
local r = redis.call('GET', KEYS[1])
if r == false or tonumber(r) < tonumber(ARGV[1]) then
   return -1
end
redis.call('DECRBY', KEYS[1], ARGV[1])
redis.call('INCRBY', KEYS[2], ARGV[1])
return 1
```

# 单测：500 抢 100 件，零超卖

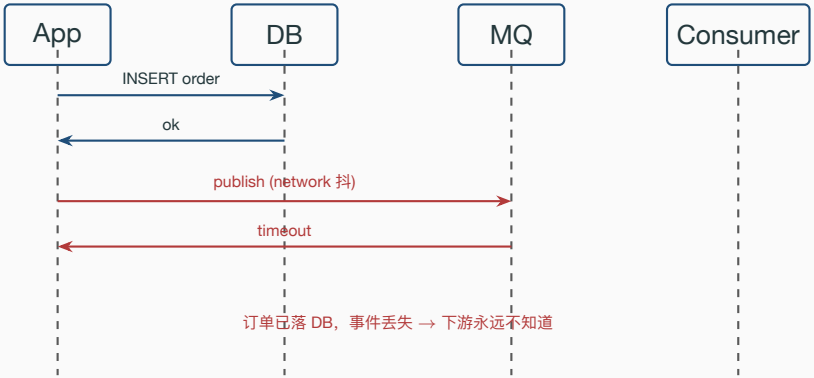
Listing 3: *repository/cache/inventory_test.go:52–82*

```go
const initial int64 = 100
const goroutines = 500
InitStock(ctx, pid, initial)
var success int64; var wg sync.WaitGroup
for i := 0; i < goroutines; i++ {
   wg.Add(1)
   go func() {
      defer wg.Done()
      if err := ReserveStock(ctx, pid, 1); err == nil {
         atomic.AddInt64(&success, 1)
      }
   }()
}
wg.Wait()
// 期望: success == 100, a == 0, r == 100
```

*stressTest/REPORT.md* 单元测试 `TestInventory_NoOversellUnderConcurrency`

# 然而还有双写问题

# 写 DB 成功 / 发 MQ 失败


App
DB
MQ
Consumer
INSERT order
ok
publish (network 抖)
timeout
订单已落 DB，事件丢失 → 下游永远不知道

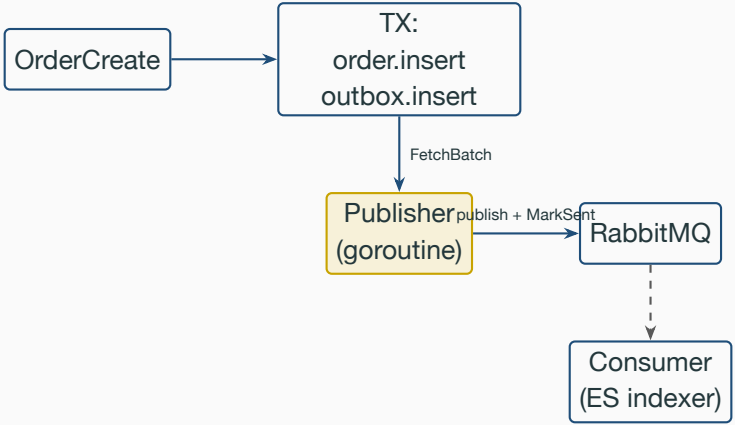
# 这就是双写问题

- 两个独立的存储系统（DB + MQ），任意一边失败都导致不一致。
- 套 2PC？性能太差且 MQ 大多不支持 XA。
- 套分布式事务？复杂度爆炸。
- 业界标准答案：**Transactional Outbox Pattern**。

*docs/blog/04-outbox-saga.md § 这就是双写问题*

# Outbox 模式

# Outbox 思路：事件先落 DB 同事务




业务和事件在同一个 DB 事务里——要么都成功，要么都失败。

# outbox_event 表结构

Listing 4: *repository/db/model/outbox.go:17–37*

```go
17 type OutboxEvent struct {
18    gorm.Model
19    AggregateType string // "order" / "product"
20    AggregateID uint // 业务实体 id
21    EventType string // "OrderCreated" / "OrderCancelled"
22    RoutingKey string // "order.created"
23    Payload string // JSON body
24    Status int // 1 pending / 2 sent / 3 dead
25    Attempts int
26    NextRetryAt time.Time
27    LastError string
28 }
```

# OrderCreate 重构：业务 + outbox 同事务

Listing 5: *service/order.go:64–78*

```go
err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
   if e := dao.NewOrderDaoByDB(tx).CreateOrder(order); e != nil {
      return e
   }
   return dao.NewOutboxDaoByDB(tx).Insert(
      "order", "OrderCreated", "order.created", order.ID,
      events.OrderCreated{
         OrderID: order.ID,
         OrderNum: order.OrderNum,
         UserID: u.Id,
         ProductID: req.ProductID,
         Num: int(req.Num),
      },
   )
})
```

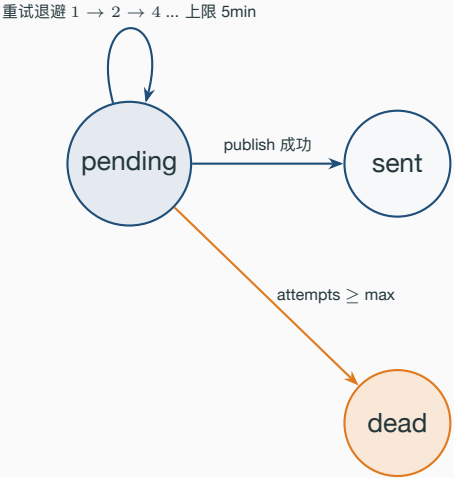
# Publisher 主循环

Listing 6: *service/outbox/publisher.go:65–86*

```go
rows, err := dao.NewOutboxDao(ctx).FetchBatch(p.cfg.BatchSize)
if err != nil { ... return }
for _, ev := range rows {
   if err := rabbitmq.PublishDomainEvent(ctx, ev.RoutingKey, []byte(ev.Payload)); err != nil {
      dao.NewOutboxDao(ctx).MarkFailed(ev.ID, ev.Attempts, p.cfg.MaxAttempts, err.Error())
      continue
   }
   dao.NewOutboxDao(ctx).MarkSent(ev.ID)
}
```

失败标记 `MarkFailed`，攻击次数到上限改为 dead；成功改为 sent。

# 失败重试 + 指数退避




*repository/db/dao/outbox.go:62–78* `MarkFailed` 指数退避

# ” 至少一次” 语义 = 消费者必须幂等

- Outbox 保证事件不丢，但不保证不重投。
- 网络抖动或 publisher 重启可能让同一事件投递多次。
- 消费端必须有去重机制（业务 id + 幂等表 / Idempotency Key）。

| 压测验证（Deck 1 复用幂等中间件） | 数值 |
|---|---|
| 同一 `Idempotency-Key` 15s 内累计请求 | **755,033** |
| 实际入库订单数（`SELECT COUNT(*) FROM order WHERE ...`） | **1** |
| 回放 p95 延迟（命中 done 态缓存） | **2.33ms** |
| 吞吐 | **50,319 RPS** |

Outbox 解决” 丢失”，幂等中间件 (Deck 1) 解决” 重复” ——两者搭配才完整。

*stressTest/REPORT.md §4.* 幂等中间件 + § 链路压测第 *4* 行

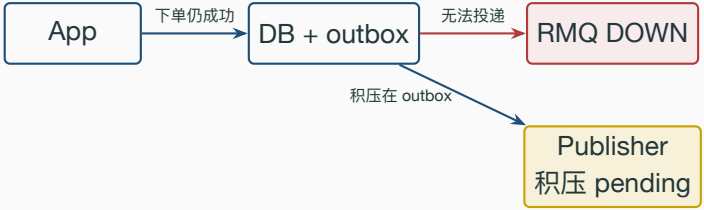
# Saga 与故障演练

# 编排式 vs 协同式 Saga + 完整链路

| 模式 | 实现 | 适用 |
| --- | --- | --- |
| 编排 (Orchestration) | 中心 Coordinator | 步骤多、需要看板 |
| 协同 (Choreography) | 事件驱动，无中心 | **本项目** |

*docs/blog/04-outbox-saga.md § 完整链路*

# 故障演练：停掉 RabbitMQ




实测：本轮压测 RMQ 全程未启动，HTTP 链路仍跑出 50,319 / 58,406 / 62,226 RPS。RMQ 恢复后 publisher 自动 drain 积压；下游 30s 内追平。业务侧零感知——这正是 Outbox 的核心价值。*stressTest/REPORT.md § 依赖：RabbitMQ 未启动 (不影响 HTTP 链路)*

# 真实事故复盘

# LogrusObj 在 InitLog 前被用

**现象**：`tryInitRabbitMQ()` 启动失败时，进程整个挂掉。

**定位过程**：

1. PR #51 (订单延迟关单) 引入 `tryInitRabbitMQ()`。
2. 该函数 `recover()` 内调用 `util.LogrusObj.Warnf(...)`。
3. `util.InitLog()` 写在 `tryInitRabbitMQ()` **之后**。
4. LogrusObj 是 nil → recover 内再次 panic → 进程退出。

**修复**：把 `util.InitLog()` 提前到所有可能 recover 的代码之前。*stressTest/REPORT.md § 已修：util.LogrusObj 在 InitLog 前被使用*

# 两条通用教训

1. **初始化顺序**：所有依赖全局对象的代码，必须在它初始化之后才能调用。
   - 把 `InitLog` / `InitTracer` / `InitMetrics` 放在 `main` 的最前。
   - 用 `init()` 函数引导循环时要看 import 顺序。
2. **recover 内禁止再 panic**：
   - recover 是最后的兜底，里面如果还能崩，进程就彻底失控。
   - 写法：`recover` 内只 print 到 stderr / 写本地文件，不依赖任何复杂组件。

*docs/blog/04-outbox-saga.md § 复盘*

# 面试 Q&A（上）

**Q1**：为什么不能 `tx.Commit()` 后再 publish MQ?

A：commit 与 publish 不在一个事务，commit 成功 publish 失败就丢事件。

**Q2**：Outbox 表挤压怎么办?

A：扩大 BatchSize；多个 publisher 实例 + SELECT FOR UPDATE 抢行；监控 lag 报警。

**Q3**：Saga 的补偿可能再失败，怎么办?

A：补偿本身也写 Outbox，失败进入死信队列由人工介入；或者设最大补偿次数兜底告警。

# 面试 Q&A（下）

**Q4**：至少一次 vs 恰好一次，消费端怎么做幂等?

A：消费端用业务 id 做去重表，或者复用 Deck 1 的 Idempotency-Key 模式。

**Q5**：” 初始化顺序” 事故你怎么定位?

A：复现栈顶 panic → 看 nil pointer → 查全局变量赋值时机 → 对照 main 顺序找到根因。

**反追问**：为什么不用 CDC (Debezium) 替代 Outbox?

A：CDC 更通用但要拉 binlog/Debezium，运维重；Outbox 是” 应用层 CDC”，简单可控。

# 代码位置一览

**双桶库存** *repository/cache/inventory.go:16–138*

**Outbox 表** *repository/db/model/outbox.go:10–37*

**Outbox DAO** *repository/db/dao/outbox.go:13–86*

**Publisher** *service/outbox/publisher.go:34–86*

**OrderCreate** *service/order.go:40–102*

**CancelUnpaid** *service/order_cancel.go:14–63*

**500 并发库存测试** *repository/cache/inventory_test.go:52–82*

故障演练：`docker compose stop rabbitmq` 后下单仍成功。

# 限流与熔断

令牌桶、滑动窗口、三态熔断器一次性梳清

gomall · 后端实战

# 今日大纲

为什么要限流

三种限流算法

分布式滑动窗口

三态熔断器

选型与边界

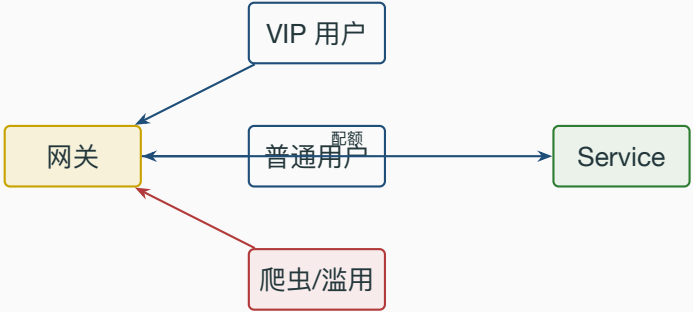
# 为什么要限流

# 误区：” 我能扛 5w RPS，不需要限流”

- 单机基线 64,254 RPS（`/ping`），听起来很安全。
- 但下游 DB / 第三方服务 / 风控不一定扛得住。
- 限流是**保护下游**，不是保护自己。

*stressTest/REPORT.md § 链路压测表第 1 行 64,254 RPS*

# 限流的四个目的




1. 隔离故障爆炸半径。
2. 保护下游容量。
3. 用户分级 (VIP $\gg$ 普通)。
4. 防滥用 / 防爬虫。

# 三种限流算法

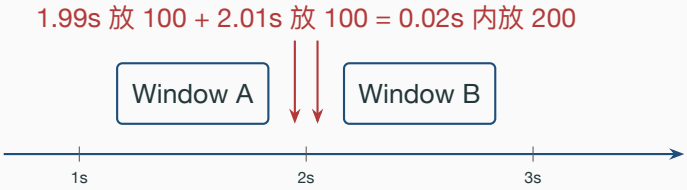
# 三种算法对比

| 算法 | 精度 | 突发 | 复杂度 |
|---|---|---|---|
| 固定窗口计数器 | 低 (临界突刺) | 不支持 | 低 |
| 令牌桶 | 中 | **支持 burst** | 低 |
| 滑动窗口 (ZSet) | 高 | 支持 | 中 |

选型：单机用令牌桶

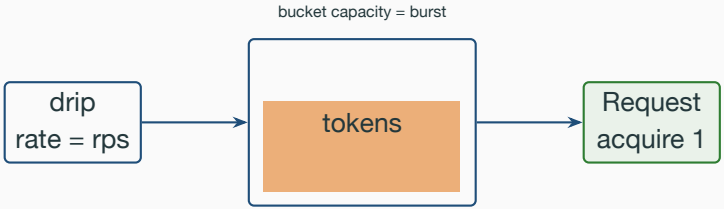
(`golang.org/x/time/rate`)；分布式用 Redis 滑动窗口。*docs/blog/05-ratelimit-circuit-breaker.md § 三种限流算法*

# 计数器临界突刺




固定窗口在边界上没有连续性，突发可以达到 2 倍阈值。

# 令牌桶：burst + 速率




令牌按速率滴入，请求消耗

令牌——短时突发允许 burst 个，长时受 rps 限制。

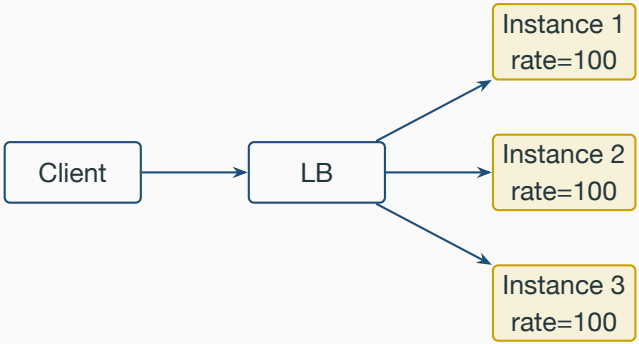
# 单机令牌桶中间件

Listing 1: *middleware/ratelimit.go:22–49*

```go
func TokenBucket(rps rate.Limit, burst int) gin.HandlerFunc {
   var mu sync.Mutex
   limiters := make(map[string]*rate.Limiter)
   get := func(ip string) *rate.Limiter {
      mu.Lock(); defer mu.Unlock()
      l, ok := limiters[ip]
      if !ok { l = rate.NewLimiter(rps, burst); limiters[ip] = l }
      return l
   }
   return func(c *gin.Context) {
      if !get(c.ClientIP()).Allow() {
         c.JSON(200, gin.H{"status": e.ErrRateLimitExceeded, ...})
         c.Abort(); return
      }
      c.Next()
   }
}
```

# 分布式滑动窗口

# 单机桶的硬伤




3 个实例各自限 100 RPS → 全局放过 300 RPS。

单机算法无法对外承诺全局阈值，必须把状态外置到 Redis。

# ZSet 滑动窗口设计

ZSet: `rl:scope:user`

member: req_id
score: ts (ms)

ZREMRANGEBYSCORE 删除 < now-window 的；ZCARD 计数；ZADD 入新点

- ZSet 按 score (时间戳) 排序，天然支持区间删除。
- ZCARD O(1) 拿当前窗口内请求数。
- 比 List 灵活：精确删除过期点而不是按头尾。

# 滑动窗口 Lua 脚本

Listing 2: *repository/cache/ratelimit.go:18–34*

```lua
local key = KEYS[1]
local window_ms = tonumber(ARGV[1])
local limit = tonumber(ARGV[2])
local now_ms = tonumber(ARGV[3])
local member = ARGV[4]

redis.call('ZREMRANGEBYSCORE', key, 0, now_ms - window_ms)
local count = redis.call('ZCARD', key)
if count >= limit then
   return {0, count}
end

redis.call('ZADD', key, now_ms, member)
redis.call('PEXPIRE', key, window_ms)
return {1, count + 1}
```

# 中间件：scope + ByUser

Listing 3: *middleware/ratelimit.go:59–106*

```go
func SlidingWindow(opt SlidingWindowOption) gin.HandlerFunc {
   return func(c *gin.Context) {
      identifier := c.ClientIP()
      if opt.ByUser {
         if u, err := ctl.GetUserInfo(c.Request.Context()); err == nil {
            identifier = formatUint(u.Id)
         }
      }
      key := cache.RateLimitKey(opt.Scope, identifier)
      nowMS := time.Now().UnixMilli()
      member := formatInt(nowMS) + "-" + randomHex()
      allowed, _, _ := cache.SlidingWindowAllow(c.Request.Context(),
         key, opt.Window.Milliseconds(), opt.Limit, nowMS, member)
      if !allowed { c.JSON(200, gin.H{"status": e.ErrRateLimitExceeded}); c.Abort(); return }
      c.Next()
   }
}
```

# 压测：精度误差 < 3%

**场景**：30 VU × 15s 打 `/skill_product/skill`, `Limit=3 Window=1s ByUser=true`。

| 指标 | 数值 |
|---|---|
| 被限流 (status=70001) | **781,624** |
| 通过 | **46** |
| 期望 (15s × 3/s) | 45 |
| 误差 | ≈ 2.2% |

*stressTest/REPORT.md §5.* 限流中间件

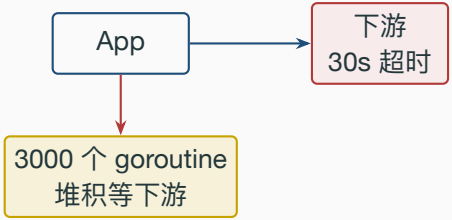
# ZSet 性能基线

- ZADD / ZREMRANGEBYSCORE / ZCARD 都是 O(logN)，N = 当前窗口内请求数。
- 1000 RPS 窗口 N $\approx$ 1000，单 op $<$ 10$\mu$s。
- Redis 单节点 10w QPS 完全扛得住分布式限流场景。

*stressTest/REPORT.md § 链路压测第 7 行：秒杀 + 限流 52,082 RPS / p95 1.24ms*

# 三态熔断器

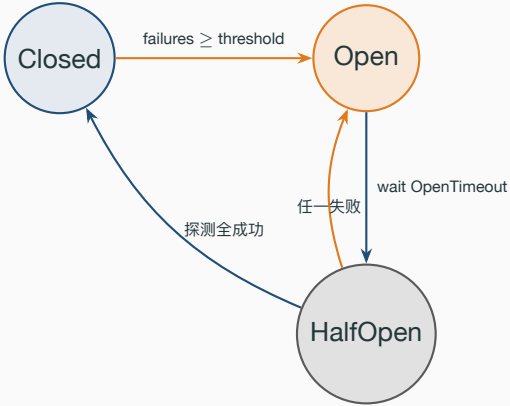
# 熔断的痛点：goroutine 堆积




下游变慢但不报错时最危险：上游 goroutine 持续堆积 → 内存爆 → 进程崩。

熔断的核心：**下游已经病了，就别再喂请求过去。**

# 三态机：Closed / Open / HalfOpen




色盲友好配色：蓝 = 正常，橙 = 告警，灰 = 过渡。HalfOpen 不是”重新接受全部”，而是**放少量探测**。

# allow(): 入口判定

Listing 4: *middleware/circuitbreaker.go:76–106*

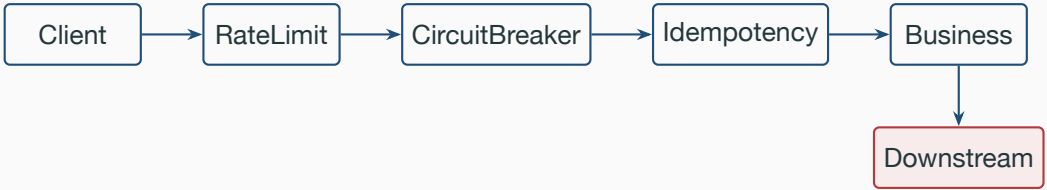
```go
func (cb *circuitBreaker) allow() error {
   switch circuitState(cb.state.Load()) {
   case stateClosed:
      return nil
   case stateOpen:
      if time.Now().UnixNano()-cb.openedAt.Load() < int64(cb.opt.OpenTimeout) {
         return errCircuitOpen
      }
      // OpenTimeout 到点：切到 HalfOpen 放探测
      cb.state.Store(int32(stateHalfOpen)); fallthrough
   case stateHalfOpen:
      if cb.halfOpenReq.Add(1) > cb.opt.HalfOpenMaxReq {
         return errCircuitOpen
      }
      return nil
   }
}
```

# report(): 出口反馈

Listing 5: *middleware/circuitbreaker.go:108–128*

```go
func (cb *circuitBreaker) report(failed bool) {
   switch circuitState(cb.state.Load()) {
   case stateClosed:
      if failed && cb.failures.Add(1) >= cb.opt.FailureThreshold {
         cb.tripOpen()
      } else if !failed { cb.failures.Store(0) }
   case stateHalfOpen:
      if failed {
         cb.tripOpen()
      } else if cb.halfOpenReq.Load() >= cb.opt.HalfOpenMaxReq {
         cb.state.Store(int32(stateClosed))
         cb.failures.Store(0)
      }
   }
}
```

# 中间件链顺序




- 限流在最外：阻挡总量。
- 熔断在中间：保护下游。
- 幂等在业务前：处理重复。

# 熔断器单测：三态都 PASS

| 用例 | 验证 | 结果 |
|---|---|---|
| `OpensAfterFailures` | Closed → Open | PASS |
| `HalfOpenToClosed` | HalfOpen 全成功 → Closed | PASS |
| `HalfOpenFailReturnsToOpen` | HalfOpen 失败 → Open | PASS |

*stressTest/REPORT.md* 单元测试表 + *middleware/circuitbreaker_test.go*

# 选型与边界

# 选型决策树

| 场景 | 工具 | 备注 |
| --- | --- | --- |
| 单服务局部限流 | 令牌桶 (`x/time/rate`) | 内存内，毫秒级 |
| 多实例全局限流 | Redis 滑动窗口 | 状态外置，精度高 |
| 保护慢依赖 | 三态熔断器 | 关注失败率不是绝对值 |
| 跨语言统一治理 | API Gateway / Sentinel | 业务侵入小 |

限流是**入口**治理，熔断是**出口**治理——边界要分清。

# 面试 Q&A（上）

**Q1**：令牌桶为什么允许” 突发”？业务上需要吗？

A：bucket capacity 给” 短时尖峰” 留缓冲，符合真实流量的脉冲特性。

**Q2**：单机限流 vs 分布式限流，分别用什么？

A：单机用进程内令牌桶；多实例必须用 Redis 滑动窗口或令牌桶。

**Q3**：滑动窗口为什么用 ZSet 不用 List？

A：ZSet 支持按 score 范围删 + O(1) 计数；List 只能按头尾，无法精确按时间清理。

# 面试 Q&A（下）

**Q4**：熔断器的 half-open 探测请求数怎么选?

A：3–5 个起步：太少噪声大，太多保护失效；按 P99 失败率反推。

**Q5**：限流后返回 429 还是 200+ 业务码?

A：移动端为兼容老网关常用 200 + status=70001；标准 RESTful 用 429 +`Retry-After`。

**反追问**：限流值怎么设?

A：压测找到 SLA 拐点 × 0.7，留余量；再按业务高峰 / 平峰区分。

# 代码位置一览

| | |
|---|---|
| **令牌桶** | *middleware/ratelimit.go:22–49* |
| **滑动窗口 mw** | *middleware/ratelimit.go:59–106* |
| **滑动窗口 Lua** | *repository/cache/ratelimit.go:18–34* |
| **熔断 allow** | *middleware/circuitbreaker.go:76–106* |
| **熔断 report** | *middleware/circuitbreaker.go:108–128* |
| **压测脚本** | *stressTest/seckill_rate_limit.js* |

复现：`k6 run stressTest/seckill_rate_limit.js`